Certain guards start with is_, such as is_number, is_bitstring, is_map, is_list, and so on. These are used as a form of type checking. They only are checked at runtime, so it's not going to offer compile errors to you, but it's incredibly useful to guarantee that your functions will only operate on certain types of data.

The functions that can be used with guards are limited. This is for your own good though—guards are guaranteed to not mutate data and their performance can be optimized. The Elixir documentation[2] has a great breakdown of available guards, limitations, and how to write your own guards.

Let's look at function overloading next.

Overloading with Pattern Matching

A function in Elixir can be defined with multiple function heads—each function head is a definition of that function for a specific pattern. When a function is called, the first function that matches the provided arguments is used. So, you can define my_func/2 multiple times with different patterns.

Let's write a to_bool/1 function that works with multiple function heads. In your examples project from the previous chapter, create lib/examples/patterns/boolean.ex:

elixir_examples/lib/examples/patterns/boolean.ex

```
Line 1  defmodule Examples.Patterns.Boolean do
        @false_s ["", "undefined", "false", "nil", "null", "-0", "0", "no", "off"]
        
        def to_bool(bool) when is_boolean(bool), do: bool
5
        def to_bool(0), do: false
        
        def to_bool(nil), do: false
        
10      def to_bool(str) when is_bitstring(str) and str in @false_s, do: false
        
        def to_bool(str) when is_bitstring(str) do
          String.downcase(str) not in @false_s
        end
15
        def to_bool(_), do: true
      end
```

Run test values through it to see it working:

```
$ iex -S mix
iex> import Examples.Patterns.Boolean

iex> to_bool(0) # false
iex> to_bool("0") # false
```

2. https://hexdocs.pm/elixir/guards.html